

UNIT III: Inheritance, types of inheritance, super keyword, final keyword, overriding and abstract class. Interfaces, creating the packages, using packages, importance of CLASSPATH and java.lang package. Exception handling, importance of try, catch, throw, throws and finally block, user defined exceptions.

Inheritance:

The process by which one class acquires the properties i.e., variables, methods, constructors, sub classes etc. from another class is called as inheritance (base class → derived class)

(or)

It is a process of creating a new class from the existing class is known as Inheritance.

Points to be remembered:

- ✓ Inheritance is the most important and useful feature of OOP.
- ✓ The **existing class** is known as **parent class or base class or super class**.
- ✓ The **new class** is known as **child class or derived class or sub class**.
- ✓ The **Parent class** is the class which provides features to another class.
- ✓ The **Child class** is the class which receives features from another class.
- ✓ It can be done by the **extends** keyword.
- ✓ It follows **IS-A** relationship.

Uses / Advantages:

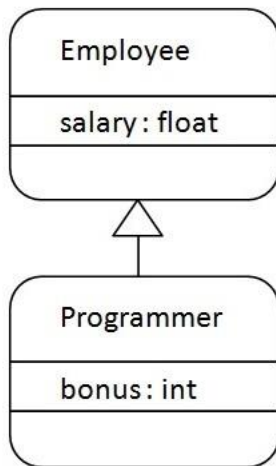
- ✓ It allows us to reuse of code to improve the performance.
- ✓ For Method Overriding (to achieve runtime polymorphism).
- ✓ we can save time and effort.
- ✓ We can extend the class with our own properties.
- ✓ The derived class can also extend the properties of base class to generate more dominant objects.
- ✓ The same base class is used for more derived classes.
- ✓ When a class is derived from more than one class, the derived classes have similar properties to those of base classes.

Disadvantages:

- ✓ Complicated.
- ✓ Invoking methods creates overhead to the compiler.

Syntax:

```
class <ChildClassName> extends <ParentClassName>
{
    //Implementation of child class
}
```

Example:

As displayed in the figure, Programmer is the subclass and Employee is the superclass. The relationship between the two classes is **Programmer IS-A Employee**. It means that Programmer is a type of Employee.

Types of Inheritance:

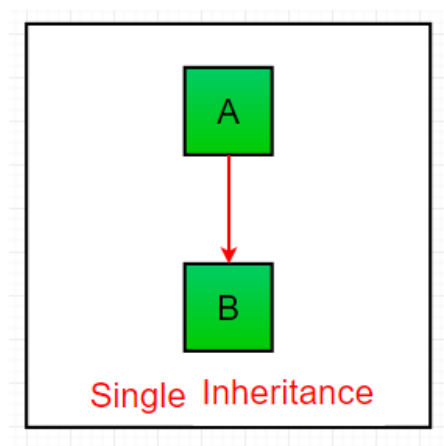
- ✓ There are 3 types of inheritances in Java
 - Single inheritance
 - Multi-level inheritance
 - Hierarchical inheritance

Note:

Multiple inheritance and Hybrid inheritance can't achieve directly but can achieve through **interface**.

Single inheritance:

It is a process of creating / deriving a class from single base class is known as **Single Inheritance**.



- ✓ The class A serves as a base class for the derived class B

Example:

```

class A{
    int a=10;
    void show(){
        System.out.print(a);
    }
}
class B extends A{
    int b=20;
    void display(){
        System.out.print(b);
    }
    void sum(){
        System.out.print(a+b);
    }
}

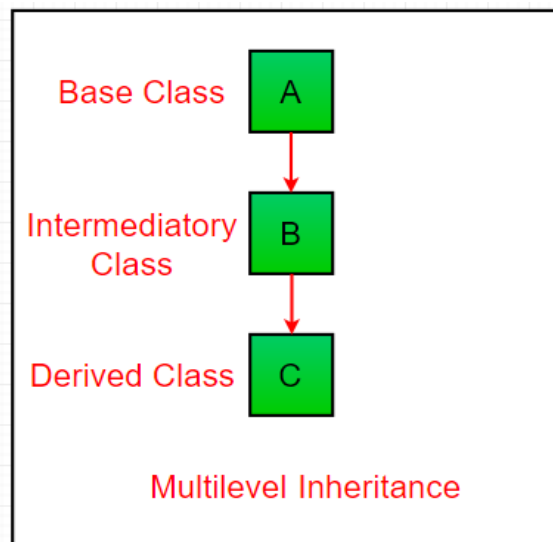
class SingleInheritance
{
    public static void main(String a[])
    {
        B b = new B();
        b.show();
        b.display();
        b.sum();
    }
}

```

Output:
10 20 30

Multilevel inheritance:

It is a process of creating / deriving a class from already derived class is known as **Multilevel Inheritance**.



- ✓ The class A serves as a base class for the derived class B, which in turn serves as a base class for the derived class C.
- ✓ C class can directly access the variables and methods of A, B and C.

Example:

```

class A{
    int a=10;
    void dispa(){
        System.out.println("A Class");
    }
class B extends A{
    int b=20;
    void dispb(){
        System.out.println("B Class");
    }
class C extends B{
    int c=30;
    void dispcc(){
        System.out.println("C Class");
    }
}

```

```

class MultiLevelInheritance{
    public static void main(String args[]){
        C obj = new C();
        System.out.println("C Class Variable="+obj.c);
        obj.dispc();
        System.out.println("B Class Variable="+ obj.b);
        obj.dispb();
        System.out.println("A Class Variable="+ obj.a);
        obj.dispa();
    }
}

```

Output:

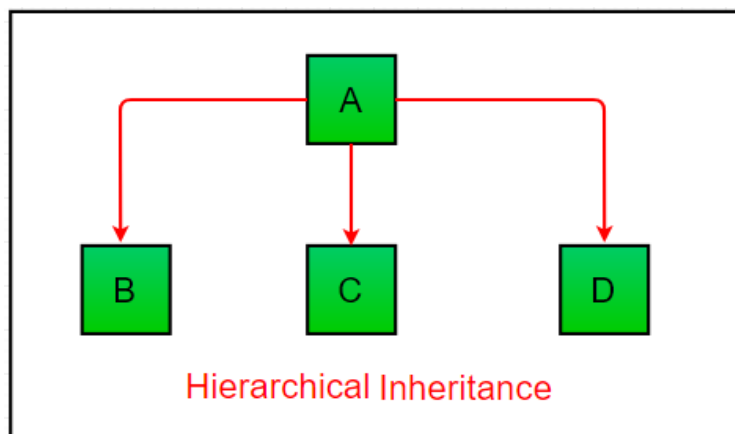
```

C Class Variable=30
C Class
B Class Variable=20
B Class
A Class Variable=10
A Class

```

Hierarchical Inheritance:

It is a process of creating / deriving Multiple classes from single base class is known as **Hierarchical Inheritance**.



- ✓ The class A serves as a base class for the derived class B, C and D.
- ✓ Using B Class Object, we can't access C class properties and D class properties.
- ✓ Using C Class Object, we can't access B class properties and D class properties.
- ✓ Using D Class Object, we can't access B class properties and C class properties.

Example:

```

class A{
    int a=10;
    void dispa(){
        System.out.println("Parent");
    }
class B extends A{
    int b=20;
    void dispb(){
        System.out.println("Child1");
    }
}
class C extends A{
    int c=30;
    void dispC(){
        System.out.println("Child2");
    }
}

```

```

class HierarchicalInheritance{
    public static void main(String args[]){
        B obj1= new B();
        C obj2 = new C();
        System.out.println("B Class Variable="+obj1.b);
        obj1.dispb();
        System.out.println("A Class Variable="+
        obj1.a);    obj1.dispa();
        System.out.println("C Class Variable="+obj2.c);
        obj2.dispc();
        System.out.println("A Class Variable="+
        obj2.a);    obj2.dispa();
    }
}

```

Output:

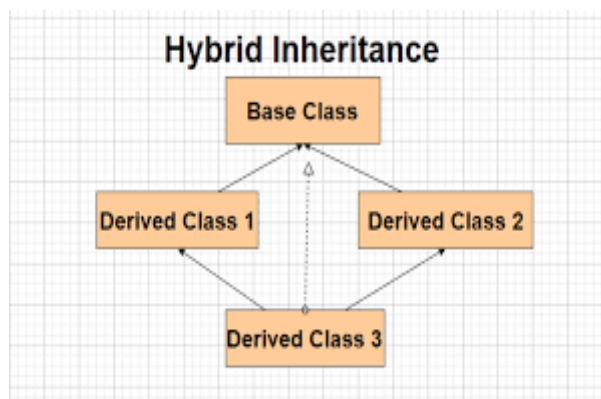
```

B Class Variable=20
Child1
A Class Variable=10
Parent
C Class Variable=30
Child2
A Class Variable=10
Parent

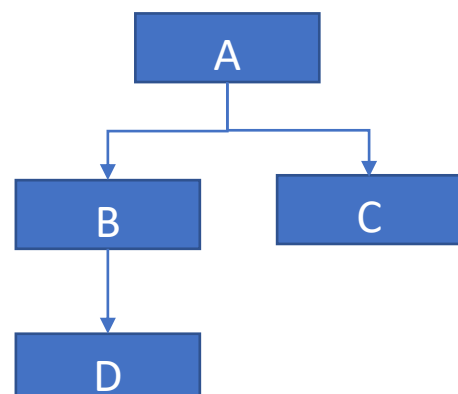
```

Hybrid Inheritance

- ✓ It is a combination of more than one type of inheritance
- ✓ If the combination consists of multiple inheritance not possible to implement



✗ Invalid



✓ Valid

Example:

```

class A
{
    int a;
}
class B extends A
{
    int b;
}
class C extends A{
    int c;
    C(){
        a=10; c=20;
    }
    void sum()
    {
        System.out.println ("Sum="+a+c));
    }
}
class D extends B{
    int d;
    D()
    {
        a=10; b=20; d=30;
    }
    void mul()
    {
        System.out.println ("Mul="+a*b*d));
    }
}
class Hybrid{
    public static void main(String args[]){
        C obj1= new C();
        D obj2= new D();
        obj1.sum();
        obj2.mul();
    }
}

```

```

class HybridInheritance {
    public static void main(String args[]){
        C obj1= new C();
        D obj2= new D();
        obj1.sum();
        obj2.mul();
    }
}

```

Output:

Sum=30

Mul=6000

Method Overriding:

If subclass (child class) has the same method as declared in the parent class, it is known as **method overriding in Java**.

(or)

Method overriding is used to provide the specific implementation of a method which is already provided by its superclass.

Imp Points:

- Method overriding is used for runtime polymorphism.
- Base class and Derived class should have the same method name (preference given to sub class)
- Both the methods should have same return type, scope and signature (no., order of parameters & Type of parameters)
- Private, Static and Final methods can't be override.

Example:

```
class Parent{
    void display()
    {
        System.out.println("PARENT CLASS");
    }
}
class Child extends Parent{
    void display()
    {
        System.out.println("CHILD CLASS");
    }
}
class Override
{
    public static void main(String args[])
    {
        Child c = new Child();
        c.display();
    }
}
```

Output:

CHILD CLASS

Method Overriding vs Method Overloading:

- ✓ Implementing the same thing with different ways i.e., Polymorphism

S. No	Method Overloading	Method Overriding
1	Can be achieved compile time polymorphism	Can be achieved run time polymorphism
2	Static binding / Early binding	Dynamic binding / Late binding
3	Implemented within a single class	Implemented in two different classes
4	No need of inheritance to achieve this	With the help of inheritance to achieve this
5	Used to increase readability of program	Used to provide different implementation of super class method
6	Method name should be same	Method name should be same
7	Signature must be different	Signature must be same
8	Return type can be same or different	Return type should be same
9	Static methods can be overloaded	Static methods can't be overridden
10	Private methods can be overloaded	Private methods can't be overridden
11	Final methods can be overloaded	Final methods can't be overridden

Super Keyword:

- It can be used to access the variables, methods, constructors of base class into derived class
- This will be used in derived class and the properties should be same in both the classes
- By using super.(property name), the derived class can access the properties of base class
- By default, constructor will be executed first

Example - 1:

```

class Parent
{
    int a=40;
    void display()
    {
        System.out.println("PARENT CLASS");
    }
}

class Child extends Parent
{
    int a=30;

```



```
void display()
{
    System.out.println ("CHILD CLASS");
    System.out.println (a);
    System.out.println(super.a);
    super.display();
}
}
class SuperDemo
{
    public static void main(String args[])
    {
        Child c = new Child();
        c.display();
    }
}
```

Output:

```
CHILD CLASS
30
40
PARENT CLASS
```

Example-2:**Super Keyword with Default Constructor:**

```
class Parent{
    Parent()
    {
        System.out.println("Parent class Constructor");
    }
}
class Child extends Parent{
    Child()
    {
        // invoke or call parent class constructor
        super();
        System.out.println("Child class Constructor");
    }
}
```

```
class Test{  
    public static void main(String[] args)  
    {  
        Child c = new Child();  
    }  
}
```

Output:**Parent class Constructor****Child class Constructor****Super Keyword with parameterized Constructor:**

```
class Base{  
    Base(int i){  
        System.out.println ("Base class Constructor Value="+i);  
    }  
}  
  
class Derived extends Base{  
    Derived(){  
        super(100);  
        System.out.println("Derived class Constructor");  
    }  
}  
  
class Test{  
    public static void main(String[] args)  
    {  
        Derived obj = new Derived();  
    }  
}
```

Output:**Base class Constructor Value=100****Derived class Constructor**

Final Keyword:

In Java, the **final** keyword is used to denote constants. It can be used with **variables, methods, and classes**.

Once any entity (variable, method or class) is declared final, it can be assigned only once. That is,

- the final variable cannot be reinitialized with another value
- the final method cannot be overridden
- the final class cannot be extended

final variable: If a variable can be defined with final then that variable value cannot be changed during the scope of program i.e., constant value.

Example-1 :

```
class FinalDemo{
    final int i=10;
    void finalDemo(){
        i=i+10;
        System.out.println("Value of i: "+i);
    }
}
class FinalTest{
    public static void main(String args[])
    {
        FinalDemo obj=new FinalDemo();
        obj.finalDemo();
    }
}
```

Output:

can't assign a value to final variable i

Example-2:

```
class Sample{
    final int a=10;
    Sample()
    {
        a=20;
    }
}
class Test{
    public static void main(String args[])
    {
        Sample obj=new Sample();
    }
}
```

Output:

can't assign a value to final variable a

final method: If a method can be defined with final then that method cannot be overridden

Example:

```
class Parent{
    final void m1()
    {
        System.out.println("Parent Class");
    }
}
class Child extends Parent{
    void m1(){
        System.out.println("Child Class");
    }
}
```

```
class FinalTest{
    public static void main(String args[]){
        Child obj=new Child();
        obj.m1();
    }
}
```

Output:

m1() in Child can't override m1() in Parent

final class: If a class can be defined with final then that class cannot be inherited.

Example: <pre>final class Parent{ void m1() { System.out.println("Parent Class"); } } class Child extends Parent{ void m1(){ System.out.println("Child Class"); } }</pre>	<pre>class FinalTest{ public static void main(String args[]){ Child obj=new Child(); obj.m1(); } }</pre> Output: cannot inherit from final Parent
---	---

Abstraction in Java: **Abstraction** is a process of hiding the implementation details and showing only functionality to the user.

Ex: sms, os, apps..etc.

There are two ways to achieve abstraction in java

- Abstract class
- Interface

Abstract class

- A class which is declared with the abstract keyword is known as an abstract class in Java.
- It can have Variables, abstract and non-abstract methods (method with the body).
- If a method is declared with abstract keyword then it is called as abstract method.
- abstract method contains only declarations i.e., it doesn't provide definition / implementation of a method.
- Implementation of abstract method can be written in derived class only
- Objects can't be created / instantiated for the abstract class and can be created for the concrete class.
- If a class is having atleast one abstract method then that class is called abstract class & it may also have normal methods.
- If a class contains a complete definition for all methods, then that class is called a concrete class.
- It can have constructors and static methods also.
- It can have final methods which will force the subclass not to change the body of the method.

Example:

<pre> abstract class A { abstract void display(); } abstract class B extends A{ void display() { System.out.println("Abstract Method in A"); } abstract void show(); } class C extends class B{ void show(){ System.out.println("Show Method in B"); display(); } } </pre>	<pre> class AbstractDemo{ public static void main(String args[]){ C c = new C(); c.show(); c.display(); } } </pre> Output: Show Method in B Abstract Method in A
--	---

Abstract Class vs Concrete Class vs Interface:

S. No	Abstract Class	Concrete Class
1	It is declared using abstract keyword	No need of access modifier
2	It is not possible to create object for abstract class	We can create object for concrete class
3	Abstract class contains abstract methods and concrete methods also	Concrete class contains only concrete methods but not abstract methods
4	Abstract class can't be declared as final	Concrete class can be declared as final

S. No	Abstract Class	Interface
1	It doesn't provide full abstraction i.e., It may contains concrete methods also	It provides full abstraction i.e., It contains only abstract methods
2	We can't implement multiple inheritance	We can implement multiple inheritance
3	Every method present in abstract class need not be public & abstract	Every method present in interface is by default public & abstract

4	Every variable present in abstract class need not be public, static & final	Every variable present in interface is always public, static & final
5	Not necessary to initialize variables during declarations in abstract class	We should perform variable initialization during declaration otherwise will get compilation error

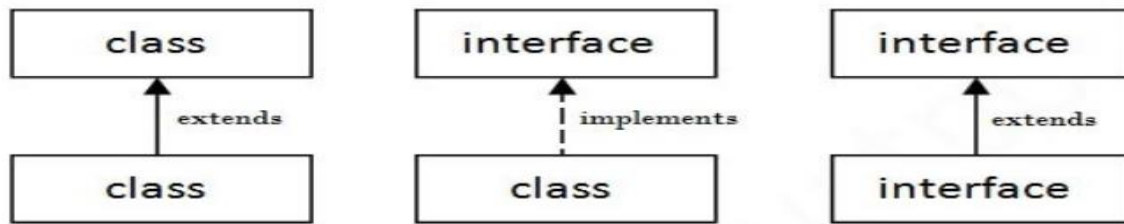
Interfaces:

- Like a class, an interface can have methods and variables, but the methods declared in an interface are by default public abstract (only method signature, no body)
- An interface cannot be instantiated, but we can create a reference for interface.
- A class that implements an interface must implement all the methods declared in the interface.
- To implement interface use **implements** keyword.
- If a class implements an interface and does not provide method bodies for all methods specified in the interface, then the class must be declared as abstract.
- If the subclass is overriding all the methods of an interface then it is called as implementation class.
- A class can implement any number of interfaces.
- By default, each and every method in the interface is a public abstract.
- By default every variable in interface acts like public static final.
- We cannot create constructors in interface.
- We can use default method in interface. (Java 8)
- We can use static method in interface.(Java 9)
- Using interface we can achieve multiple inheritance.
- To declare an interface, use interface keyword

Syntax:

```
interface interface_name {
    // declare variables        // By default public static final    int a=10;
    // declare methods          // By default public abstract
}
```

The relationship between classes and interfaces



Example-1:

```

interface Interface_Example{
    void test();
}
class Test implements Interface_Example{
    public void test(){
        System.out.println ("Interface Method Implemented");
    }
    public static void main(String args[]){
        Test p = new Test();
        p.test();
    }
}

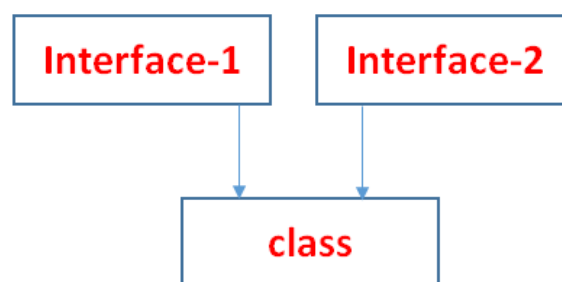
```

Output:

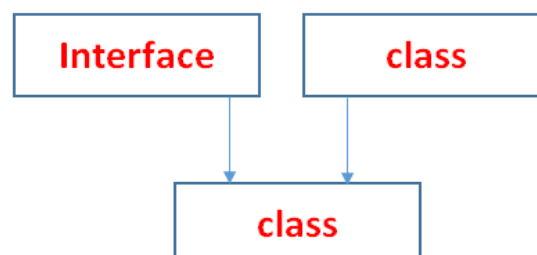
Interface Method Implemented

Multiple Inheritance:

1)



2)



Example-2:

```
interface A{
    int a=10;
    void show();
}
interface B{
    int b=20;
    void show();
}
class C implements A,B{
    int c;
    public void show(){
        c=A.a+B.b;
        System.out.println("A.a="+A.a);
        System.out.println("B.b="+B.b);
        System.out.println("C="+c);
    }
}
class InterfaceDemo{
    public static void main(String args[]){
        C obj = new C();
        obj.show();
    }
}
```

Output:

A.a=10
B.b=20
C=30

Example – 3:

```
interface A{
    int a=10;
    void show();
}
interface B{
    int b=20;
    void show();
}
class C implements A,B{
    int c;
    public void show()
    {
        c=a+b;
        System.out.println("A="+a);
        System.out.println("B="+b);
        System.out.println("C="+c);
    }
}
public class InterfaceDemo{
    public static void main(String args[]){
        C obj = new C();
        obj.show();
    }
}
```

Output:

A=10
B=20
C=30

Example-2:

```
interface A
{
    int a=10;
    public void test();
}
class B
{
    int b=20;
    public void show()
    {
        System.out.println("class Method Implemented");
        System.out.println("b=" + b);
    }
}
class Sample extends B implements A
{
    public void test()
    {
        System.out.println("Interface-A Method Implemented");
        System.out.println("a=" + a);
    }
}
public class Interface_Demo
{
    public static void main(String args[])
    {
        Sample p = new Sample();
        p.test();
        p.show();
    }
}
```

INPUT / OUTPUT:

```
Interface-A Method Implemented
a=10
class Method Implemented
b=20
```

Exception Handling:

- ✓ Exception is an unwanted event that interrupts the normal flow of the program (or) runtime error which will occurs during program execution
- ✓ If any error occurred, the remaining code will not execute and terminate the program abnormally
- ✓ Exception handling allows us to handle the runtime errors caused by exceptions
- ✓ Used to raise an exception, handle an exception and avoiding abnormal termination of a program
- ✓ We use 5 keywords in this i.e., try, catch, throw, throws, finally
- ✓ Exceptions are handled with the help of try and catch blocks
- ✓ Try and catch blocks are interrelated i.e., without try we can't use catch and vice versa.

Hierarchy of Java Exception classes:

The java.lang.Throwable class is the root class of Java Exception hierarchy inherited by two subclasses: Exception and Error. The hierarchy of Java Exception classes is given below:



Types of Java Exceptions:

There are mainly two types of exceptions: checked and unchecked. An error is considered as the unchecked exception. However, according to Oracle, there are three types of exceptions namely:

1. Checked Exception
2. Unchecked Exception
3. Error

1. Checked (Compile Time) Exception:

- ✓ The classes which directly inherit **Throwable** class except **RuntimeException** and **Error** are known as checked exceptions
- ✓ Checked exceptions are checked at compile-time
- ✓ **Ex:** IOException, SQLException etc...

2. Unchecked (Run Time)Exception:

- ✓ The classes which inherit **RuntimeException** are known as unchecked exceptions
- ✓ Unchecked exceptions are not checked at compile-time, but they are checked at runtime
- ✓ **Ex:** ArithmeticException, ArrayIndexOutOfBoundsException, NullPointerException, NumberFormatException etc...

3. Error

- ✓ Error is irrecoverable and it occurs due to problem with computer & software
- ✓ **Ex:** OutOfMemoryError, VirtualMachineError, AssertionError etc...

Java Exception Keywords:

Keyword	Description
try	"try" keyword is used to specify a block where we should place exception code The try block must be followed by either catch or finally. It means, we can't use try block alone
catch	"catch" block is used to handle the exception It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later
finally	"finally" block is used to execute the important code of the program It is executed whether an exception is handled or not
throw	"throw" keyword is used to throw an exception implicitly or explicitly from a method or any block of code.
throws	"throws" keyword is used to declare exceptions. It doesn't throw an exception It specifies that there may occur an exception in the method & always used with method signature.

Built-in exceptions:**1. ClassNotFoundException**

This Exception is raised when we try to access a class whose definition is not found

2. FileNotFoundException

This Exception is raised when a file is not accessible or does not open

3. IOException

It is thrown when an input-output operation failed or interrupted

4. InterruptedException

It is thrown when a thread is waiting, sleeping or doing some processing and it is interrupted

5. ArithmeticException

It is thrown when an exceptional condition has occurred in an arithmetic operation

6. ArrayIndexOutOfBoundsException

It is thrown to indicate that an array has been accessed with an illegal index. The index is either negative or greater than or equal to the size of the array

7. NullPointerException

This exception is raised when referring to the members of a null object. Null represents nothing

8. NumberFormatException

This exception is raised when a method could not convert a string into a numeric format.

9. NoSuchFieldException

It is thrown when a class does not contain the field (or variable) specified

10. NoSuchMethodException

It is thrown when accessing a method which is not found.

11. RuntimeException

This represents any exception which occurs during runtime.

12. StringIndexOutOfBoundsException

It is thrown by String class methods to indicate that an index is either negative than the size of the string

Java try-catch block:

- ✓ "try" keyword is used to specify a block where we should place exception code
- ✓ The try block must be followed by either catch or finally. It means, we can't use try block alone

Syntax:

```
try
{
    //code that may throw an exception
}
catch (Exception_class_Name ref)
{
    // handle the exception
}
```

Syntax:

```
try
{
    //code that may throw an exception
}
finally
{
    // important code
}
```

Example-1:

```
public class ExceptionHandlingDemo {

    public static void main(String[] args) {

        int x,y;
        int a=10,b=5,c=5;
        x=a/(b-c);
        System.out.println(x);
        y=a/(b+c);
        System.out.println(y);
    }
}
```

Output:

Exception in thread "main" [java.lang.ArithmeticException: / by zero](#)

Example-2:

```
public class ExceptionHandlingDemo1 {
    public static void main(String[] args) {
        int x,y;
        int a=10,b=5,c=5;
        try
        {
            x=a/(b-c);
            System.out.println(x);
        }
        catch (ArithmeticException e)
        {
            //System.out.println(e);
            System.out.println("Exception handled");
        }
        y=a/(b+c);
        System.out.println(y);
        System.out.println("End of Try-Catch Block");
    }
}
```

Output:

[java.lang.ArithmeticException](#): / by zero

1

End of Try-Catch Block

Types of Unchecked (Run Time)Exceptions:

1. ArithmeticException → divide by zero
2. NullPointerException → String str = null;
System.out.println(str.length()); ✗
3. NumberFormatException → String str = "hello";
int num = Integer.parseInt(str); ✗
4. ArrayIndexOutOfBoundsException → int a[] = new int[5];
a[8] = 20; ✗

Java Multi-catch block:

A try block can be followed by one or more catch blocks. Each catch block must contain a different exception handler. So, if you have to perform different tasks at the occurrence of different exceptions, use java multi-catch block.

Points to remember:

- At a time only one exception occurs and at a time only one catch block is executed.
- All catch blocks must be ordered from most specific to most general, i.e. catch for ArithmeticException must come before catch for Exception.

Example-3:

```
public class ExceptionHandlingDemo2 {
    public static void main(String[] args) {
        try
        {
            int x=10/0;
            System.out.println(x);

            int[] a=new int[5];
            System.out.println(a[10]);

            String b="REC";
            int n= Integer.parseInt(b);

            String c=null;
            System.out.println(c.length());
        }
    }
}
```

```

        catch (ArithmeticException e)
        {
            System.out.println(e);
        }
        catch (ArrayIndexOutOfBoundsException e)
        {
            System.out.println(e);
        }
        catch (NumberFormatException e)
        {
            System.out.println(e);
        }
        catch (NullPointerException e)
        {
            System.out.println(e);
        }
        System.out.println("End of the program");
    }
}

```

Output:

[java.lang.ArithmeticException](#): / by zero

End of the program

Example-4:

```

public class ExceptionHandlingDemo2 {
    public static void main(String[] args) {
        try
        {
            int x=10/0;
            System.out.println(x);

            int[] a=new int[5];
            System.out.println(a[10]);

            String b="REC";
            int n= Integer.parseInt(b);

            String c=null;
            System.out.println(c.length());
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
        System.out.println("End of Try-Catch Block");
    }
}

```

Output:

[java.lang.ArithmeticException](#): / by zero

End of the program

Java Nested try Catch block:

In Java, using a try block inside another try block is permitted. It is called as nested try block.

For example, the inner try block can be used to handle **ArithmeticException (division by zero)**

While the outer try block can handle the **ArrayIndexOutOfBoundsException**.

Example:

```
public class NestedTry {
    public static void main(String[] args) {
        try
        {
            int a[]={10,0,30,40,50};
            try
            {
                int b=a[3]/a[1];
            }
            catch(ArithmeticException e)
            {
                System.out.println(e);
            }
            a[5]=100;
        }
        catch(ArrayIndexOutOfBoundsException e)
        {
            System.out.println(e);
        }
    }
}
```

Output:

[java.lang.ArithmeticException](#): / by zero

[java.lang.ArrayIndexOutOfBoundsException](#): Index 5 out of bounds for length 5

Usage of Java finally:

"finally" block is used to execute the important code of the program. It is executed whether an exception is handled or not

Syntax:

```
try
{
    //code that may throw an exception
}
finally
{
    // important code
}
```

Syntax:

```
try
{
    //code that may throw an exception
}
catch (Exception_class_Name ref)
{
    // handle the exception
}
finally
{
    // important code
}
```


Example -1: When an exception does not occur

```
public class FinallyDemo {  
    public static void main(String[] args) {  
        int x,y;  
        int a=10,b=5,c=5;  
        try  
        {  
            y=a/(b+c);  
            System.out.println(y);  
        }  
        finally  
        {  
            System.out.println("This is finally block");  
        }  
    }  
}
```

Output:

1

This is finally block

Example -2: When an exception occur but not handled by the catch block

```
public class FinallyDemo {  
    public static void main(String[] args) {  
        int x,y;  
        int a=10,b=5,c=5;  
        try  
        {  
            y=a/(b-c);  
            System.out.println(y);  
        }  
        catch(NullPointerException e)  
        {  
            System.out.println(e);  
        }  
        finally  
        {  
            System.out.println("This is finally block");  
        }  
    }  
}
```

Output:

This is finally block

Exception in thread "main" [java.lang.ArithmeticException: / by zero](#)
at FinallyDemo.main([FinallyDemo.java:9](#))

Example -3: When an exception occurs and is handled by the catch block

```
public class FinallyDemo {  
    public static void main(String[] args) {  
        int x,y;  
        int a=10,b=5,c=5;  
        try  
        {  
            y=a/(b-c);  
            System.out.println(y);  
        }  
        catch(ArithmeticException e)  
        {  
            System.out.println("Divided by zero error");  
        }  
        finally  
        {  
            System.out.println("This is finally block");  
        }  
    }  
}
```

Output:

Divided by zero error

This is finally block

Java throw keyword:

- ✓ The throw keyword in Java is used to implicitly or explicitly throw an exception from a method or any block of code
- ✓ We can throw either checked or unchecked exceptions in Java by throw keyword. It is mainly used to throw a custom exception.

Syntax

```
throw new exception_class("error message");
```

Example-1:

```
import java.lang.*;  
class ThrowExcep  
{  
    public static void main (String[] args) {  
        try  
        {  
            throw new ArithmeticException("My Own Exception");  
        }  
        catch(ArithmeticException e)  
        {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

```
}
```

Output:

My Own Exception

Example-2:

```
class ThrowExcep
{
    static void fun()
    {
        try
        {
            throw new NullPointerException("demo");
        }
        catch(NullPointerException e)
        {
            System.out.println("Exception Caught inside fun().");
            throw e; // rethrowing the exception
        }
    }
}

public static void main(String args[])
{
    try
    {
        fun();
    }
    catch(NullPointerException e)
    {
        System.out.println("Exception Caught in main().");
    }
}
}
```

Output:

Exception Caught inside fun().

Exception Caught in main().

User-defined Custom Exception:

- ✓ Java provides us facility to create our own exceptions which are basically derived classes of Exception class.

For example MyException in below code extends the Exception class.

Example:

```
class MyException extends Exception{
    public MyException(String s){
        // Call constructor of parent Exception
        super(s);
    }
}

public class ThrowExcep{
    public static void main(String args[]){
        try
        {
            throw new MyException("User defined exception");
        }
        catch (MyException ex)
        {
            System.out.println("Exception Caught");
            System.out.println(ex.getMessage());
        }
    }
}
```

Output:

```
Caught
User defined exception
```

Java throws:

- ✓ throws is a keyword in Java that is used in the declaration of a method to indicate that this method might throw one of the listed type exceptions.
- ✓ The caller to these methods has to handle the exception using a try-catch block.

Syntax

type method_name(parameters) throws exception_list

Example-1:

```
class ThrowsDemo {
    public static void main(String[] args)
    {
        Thread.sleep(10000);
        System.out.println("Java Programming");
    }
}
```

Output:

```
ThrowsDemo.java:4: error: unreported exception InterruptedException; must be caught or
declared to be thrown
Thread.sleep(10000);
```

Example – 2:

```
class ThrowsDemo {  
    public static void main(String[] args) throws InterruptedException  
    {  
        Thread.sleep(10000);  
        System.out.println("Java Programming");  
    }  
}
```

Output:

Java Programming

Example – 3:

```
class ThrowsDemo {  
    static void fun() throws ArithmeticException  
    {  
        System.out.println("Inside fun().");  
        throw new ArithmeticException("My Own Exception");  
    }  
    public static void main(String[] args)  
    {  
        ThrowsDemo obj = new ThrowsDemo();  
        try  
        {  
            fun();  
        }  
        catch(ArithmeticException e)  
        {  
            System.out.println("caught in main.");  
        }  
    }  
}
```

Output:

Inside fun().
caught in main.

Example – 4:

```
class ThrowsDemo {  
    static void calculate() throws ArithmeticException  
    {  
        System.out.println("Inside Calculate Method");  
        System.out.println(10/0);  
    }  
    public static void main(String args[])  
    {  
        try  
        {  
            calculate();  
        }  
    }  
}
```

```

    }
    catch(Exception e)
    {
        System.out.println(e);
    }
}
}

```

Output:

Inside Calculate Method

java.lang.ArithmeticException: / by zero

Throw vs Throws:

S. No.	Throw	Throws
1	It is used to throw an exception explicitly	It is used to declare an exception
2	void m() { throw new ArithmeticException("Hai"); }	void m() throws ArithmeticException { } }
3	Checked exceptions can't propagated	Checked exceptions can be propagated
4	throw is followed by instance	throws is followed by exception class
5	throw is used within a method	throws is used with method signature
6	We can't throw multiple exceptions	We can declare multiple exceptions

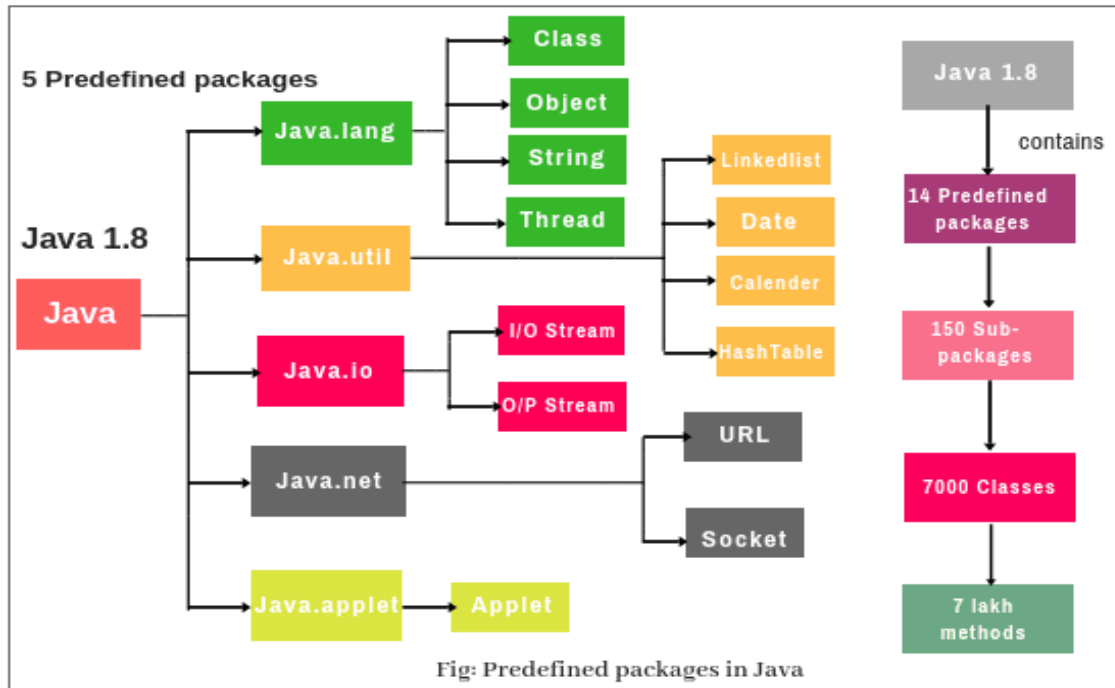
Packages in Java:

- ✓ A **java package** is a group of similar types of sub-packages, classes and interfaces.
- ✓ Packages in java can be categorized in two form, **built-in packages** and **user-defined packages**.

Built-in packages:

Predefined packages in Java are those which are developed by Sun Microsystem. They are also called built-in packages. These packages consist of a large number of predefined classes, interfaces, and methods that are used by the programmer to perform any task in his programs.

Some of the inbuilt packages in java are:



- **java.awt** : Contains classes for creating user interfaces and for painting graphics and images. Classes like *Button*, *Color*, *Event*, *Font*, *Graphics*, *Image* etc are part of this package.
- **java.io** : Provides classes for system input/output operations. Classes like *BufferedReader*, *BufferedWriter*, *File*, *InputStream*, *OutputStream*, *PrintStream*, *Serializable* etc are part of this package.
- **java.lang** : Contains classes and interfaces that are fundamental to the design of Java programming language. Classes like *String*, *StringBuffer*, *System*, *Math*, *Integer* etc are part of this package.
- **java.net** : Provides classes for implementing networking applications. Classes like *Authenticator*, *HttpCookie*, *Socket*, *URL*, *URLConnection*, *URLEncoder*, *URLDecoder* etc are part of this package.
- **java.sql** : Provides the classes for accessing and processing data stored in a database. Classes like *Connection*, *DriverManager*, *PreparedStatement*, *ResultSet*, *Statement* etc are part of this package.
- **java.util** : Contains the collections framework, some internationalization support classes, properties, random number generation classes. Classes like *ArrayList*, *LinkedList*, *HashMap*, *Calendar*, *Date*, *TimeZone* etc are part of this package.

User-defined packages:

The package which is defined by the user is called user-defined or custom package in Java. It contains user-defined classes and interfaces. Let's understand how to create a user-defined package.

Advantage of Java Package:

1. Reuse of code
2. Java package is used to categorize the classes and interfaces so that they can be easily maintained.
3. Java package provides access protection.
4. Java package removes naming collision.

Creating User-Defined Packages

Java supports a keyword called “package” which is used to create user-defined packages in Java programming. The general syntax to create a package is as:

Syntax:

package package-name;

Example:

//save as Test.java

```
package pack1;
public class Test
{
    static int a=10,b=20;
    public static void add()
    {
        System.out.println(a+b);
    }
}
```

Importing Java Package

To import java package into a class, we need to use java **import** keyword which is used to access package and its classes into the java program.

There are 3 different ways to refer to any class that is present in a different package:

1. without import the package
2. import package with specified class
3. import package with all classes

1. Without import the package

If you use fully qualified name to import any class into your program, then only that particular class of the package will be accessible in your program, other classes in the same package will not be accessible.

Example:

```
//save as Add.java
package pack2;

public class Add{

    public static void main(String[] args) {

        pack1.Test ob = new pack1.Test();

        ob.add();

    }

}
```

Output:

30

2. import package with specified class

Package can have many classes but sometimes we want to access only specific class in our program in that case, Java allows us to specify class name along with package name.

Example:

```
//save as Add.java
package pack2;

import pack1.Test;

public class Add{

    public static void main(String[] args) {

        Test ob = new Test();

        ob.add();

    }

}
```

Output:

30

3. import package with all classes

If we use **packagename.* statement**, then all the classes and interfaces of this package will be accessible but the classes and interface inside the sub-packages will not be available for use.

Example:

```
//save as Add.java
package pack2;

import pack1.*;

public class Add{
```

```
    public static void main(String[] args) {  
        Test ob = new Test();  
        ob.add();  
    }  
}
```

Output:

30

Sub package in Java:

Package inside the package is called the sub package. It should be created to categorize the package further.

Syntax:

package package-name. sub package name;

Example: // Save as Display.java

Sub package creation:

```
package pack1.sp1;  
public class Display {  
    public void display()  
    {  
        System.out.println("Sub package");  
    }  
}
```

Importing Java Package: // Save as SubpackageDemo.java

```
package pack2;  
import pack1.sp1.Display;  
public class SubpackageDemo {
```

```
    public static void main(String[] args) {  
        Display ob = new Display();  
        ob.display();  
    }  
}
```

Output:

Sub package

Static import in Java:

static import is a feature that expands the capabilities of **import** keyword. It is used to import static member of a class. We all know that static members are referred in association with its class name outside the class. Using **static import**, it is possible to refer to the static member directly without its class name.

Example without using static import:

```
public class Test
{
    public static void main(String[] args)
    {
        System.out.println(Math.sqrt(4));
    }
}
```

Output:

2.0

Example using static import

```
import static java.lang.Math.*;

public class Test
{
    public static void main(String[] args)
    {
        System.out.println(sqrt(4));
    }
}
```

Output:

2.0

Package creation in offline mode:**Example:**

```
package pack1;

class Sample{
    public static void main(String args[]){
        System.out.println("Welcome to java");
    }
}
```

To compile the class, we can use the same command that we used for package. Command is given below.

```
javac -d . Sample.java
```

To run the class stored into the created sub package, we can use below command.

```
java pack1/Sample
```

Access Modifiers in Java:

The access modifier in Java specifies the accessibility or scope of a field, method, constructor, or class. We can change the access level of fields, constructors, methods, and class by applying the access modifier on it.

There are four types of Java access modifiers:

1. **Private:** The access level of a private modifier is only within the class. It cannot be accessed from outside the class.
2. **Default:** The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.
3. **Protected:** The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.
4. **Public:** The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.

Note:

There are many non-access modifiers, such as static, abstract, synchronized, native, volatile, transient, etc. Here, we are going to learn the access modifiers only.

Understanding Java Access Modifiers:

Let's understand the access modifiers in Java by a simple table.

Access Modifier	within class	within package	outside package by subclass only	outside package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y

Wrapper classes in Java:

The **wrapper class in Java** provides the mechanism *to convert primitive into object and object into primitive*.

Since J2SE 5.0, **autoboxing** and **unboxing** feature convert primitives into objects and objects into primitives automatically. The automatic conversion of primitive into an object is known as autoboxing and vice-versa unboxing.

Primitive Type	Wrapper class
boolean	Boolean
char	Character
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double

Autoboxing:

The automatic conversion of primitive data type into its corresponding wrapper class is known as autoboxing, for example, byte to Byte, char to Character, int to Integer, long to Long, float to Float, boolean to Boolean, double to Double, and short to Short.

Example:

```
public class WrapperDemo {  
    public static void main(String args[])  
    {  
        int a=100;  
        Integer obj1 = new Integer(a);  
        Integer i=Integer.valueOf(a); //converting int into Integer explicitly  
        Integer j=a;                  //autoboxing  
        System.out.println(obj1+" "+i+" "+j);  
    }  
}
```

Output:

100 100 100